

POST and PUT JSON Requests

Send JSON to `methods.php` and receive JSON

Learning Goals

- Distinguish GET, POST, and PUT
- Read the request method and `Content-Type`
- Read a raw JSON request body from `php://input`
- Decode JSON and return a JSON response
- Recognize `400 Bad Request` and `405 Method Not Allowed`

IMPORTANT

Where Request Data Goes

Method	Typical purpose	Data location
GET	Read	URL query string
POST	Create/process	Request body
PUT	Update/replace	Request body

| Request Body Security

A request body in **POST** and **PUT** methods is not shown in the URL, but it is not automatically secret.

- Use HTTPS to encrypt it in transit, and use authentication and authorization to control who can send it and what they can do.
- Its maximum size depends on the client and server configuration.

Sending POST Requests from Clients

Start the server with this lesson's code directory as its document root:

```
cd code/3_Request_and_Response_with_PHP  
  
# Start the server  
php -S localhost:8000
```

IMPORTANT

POST Requests

- **POST** sends data to the server for processing or creating a new resource.
 - The data is usually sent in the request body rather than in the URL.
 - The body can contain form data or JSON, depending on the **Content-Type** header.
- Unlike **GET**, a **POST** request can create or change server data.

| Sending a POST Request with curl

The URL `/methods.php` selects `methods.php`.

```
curl -X POST http://localhost:8000/methods.php \  
  -H "Content-Type: application/json" \  
  -d '{"name":"Alice"}'
```

- `-X POST` selects the method.
- `-H (header)` identifies the body as JSON.
- `-d (data)` sends the request body.

Sending a POST Request with a Web Browser

In a web browser, you can create a POST request in two ways:

1. Submit an HTML `<form>` with `method="post"`.
2. Use JavaScript, such as `fetch()`.

IMPORTANT

PUT Requests

- **PUT** commonly updates or replaces an existing resource.
 - In this example, `methods.php` reads `{"name": "Alice Updated"}` from `php://input` and returns it as JSON in the `data` field.
 - It does not update a database yet.

PUT with JSON

```
curl -X PUT http://localhost:8000/methods.php \  
  -H "Content-Type: application/json" \  
  -d '{"name": "Alice Updated"}'
```

IMPORTANT

How `methods.php` Processes a Request

1. Allow only POST and PUT; otherwise return `405`.
2. Read the request method, `Content-Type`, and raw body.
3. Decode JSON, use `$_POST`, or parse a URL-encoded PUT body.
4. Return `400` if the parsed body is not an array.
5. Return the method, content type, and parsed data as JSON.

Reading code: `methods.php`

The following slides read `methods.php` top to bottom, in the order PHP executes it.

Step 1: Strict Types and Response Header

```
declare(strict_types=1);  
header('Content-Type: application/json');
```

- `declare(strict_types=1)` enables strict type checking if typed code is added.
- The response header tells the client that every response, including errors, is JSON.

Step 2: Check the Method

```
$method = $_SERVER['REQUEST_METHOD'];  
if (!in_array($method, ['POST', 'PUT'], true)) {  
    http_response_code(405);  
    header('Allow: POST, PUT');  
    echo json_encode(['error' => 'Use POST or PUT']);  
    exit;  
}
```

- For example, `-X PUT` becomes `$_SERVER['REQUEST_METHOD']`, and its value is `PUT`.
 - Only `POST` and `PUT` are allowed.
- Any other method receives `405 Method Not Allowed`, an `Allow: POST, PUT` header, and no further processing.

Step 3: Read Content Type and Body

```
$contentType = $_SERVER['CONTENT_TYPE'] ?? '';  
$isJson = str_contains($contentType, 'application/json');
```

- `-H "Content-Type: application/json"`
becomes `$_SERVER['CONTENT_TYPE']`.
 - The script uses `str_contains()` to check whether the content type includes `application/json`.

```
$rawBody = file_get_contents('php://input');
```

- `-d '{"name":"Alice Updated"}'` becomes the raw body read from `php://input`.
- `php://input` is a read-only PHP stream for the raw HTTP request body.
- `file_get_contents()` reads its contents as a string, which `json_decode()` can then convert into a PHP array.

Step 3 Note: `$isJson` Is Independent of Method

- `$isJson` can be `true` for either `POST` or `PUT`; it depends on the `Content-Type` header, not on the request method.
- When it is `true`, the JSON body must be read from `php://input`, not `$_POST`.

Step 4: Decode or Parse the Body

```
if ($isJson) {  
    $data = json_decode($rawBody, true);  
} elseif ($method === 'POST') {  
    $data = $_POST;  
} else { # $method is 'PUT'  
    parse_str($rawBody, $data);  
}
```

The script handles three cases: JSON bodies, form data in a **POST**, and URL-encoded data in a non-JSON **PUT**.

The script reads the raw body once into `$rawBody` and then chooses one branch:

1. If `$isJson` is `true`, the content type includes `application/json`, so `json_decode($rawBody, true)` converts it into a PHP array.
2. If `$isJson` is `false` and `$method` is `POST`, the script assigns the form data in `$_POST` to `$data`.
3. Otherwise, the request is a non-JSON `PUT`. `parse_str()` converts its URL-encoded raw body into `$data`.

Step 4 Example: Non-JSON PUT

For example, this request sends

`name=Alice&age=20`:

```
curl -X PUT http://localhost:8000/methods.php \  
  -H "Content-Type: application/x-www-form-urlencoded" \  
  -d "name=Alice&age=20"
```

`parse_str()` converts the body into `['name' => 'Alice', 'age' => '20']`.

Step 5: Validate the Decoded Body

```
if (!is_array($data)) {  
    http_response_code(400);  
    echo json_encode(['error' => 'Invalid request body data']);  
    exit;  
}
```

Step 5 checks that parsing produced an array.

- Invalid JSON or another data type receives **400 Bad Request**, and **exit** stops the script.

Step 6: Send the Response

```
echo json_encode([  
    'method' => $method,  
    'content_type' => $contentType,  
    'data' => $data,  
], JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES);
```

The response contains the request method, content type, and parsed data.

- `JSON_PRETTY_PRINT` formats the JSON for easier reading, and `JSON_UNESCAPED_SLASHES` keeps slashes readable.

For the earlier POST example, the input is:

```
Method: POST  
Content-Type: application/json  
Body: {"name":"Alice"}
```

The output compares with the input as follows:

```
{  
  "method": "POST",  
  "content_type": "application/json",  
  "data": {  
    "name": "Alice"  
  }  
}
```

Test the Error Responses

GET is not allowed:

```
curl -i http://localhost:8000/methods.php
```

Invalid JSON receives **400**:

```
curl -i -X POST http://localhost:8000/methods.php \  
  -H "Content-Type: application/json" \  
  -d '{bad}'
```

Key Takeaways

1. POST and PUT can carry JSON in the request body.
2. `Content-Type` describes the body format.
3. JSON must be read from `php://input` and decoded.
4. Validate decoded data before using it.
5. Return meaningful HTTP status codes for invalid requests.